



Programa Certified AI-LLM Solution Architect

Fundamentos de Arquitectura LLM

Proveedores, APIs y servicios de inferencia para IA generativa

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 25 años de experiencia
- PGP Artificial Intelligence & Machine Learning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data – IEBSchool
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialista en MLOps de la Universidad de Duke
- Estudiante de Deep Learning for Healthcare Specialization
- Director de Inteligencia Artificial e Innovacion Universidad EAN/Colombia



anrojas@universidadean.edu.co / arojaspa@outlook.com / arojaspa76



@arojaspa



<https://www.linkedin.com/in/arojaspa/>



+573005906373



CAPÍTULO 2

SESIÓN 3

3 HORAS

Proveedores y Servicios de Inferencia

Panorama de proveedores, comparativa costo/calidad y términos de licenciamiento



Directo



Multi-Cloud



Azure



GCP



AWS

Agenda — Sesión 3

Tres bloques de trabajo • 3 horas de alto impacto

01

60 min

Panorama de Proveedores de Inferencia

- ▶ OpenAI, Anthropic, Google, Meta, Mistral
- ▶ Azure OpenAI, Vertex AI, AWS Bedrock
- ▶ Directo vs Hyperscaler: ventajas y trade-offs
- ▶ Ollama: inferencia local sin costo

02

80 min

Comparativa y Benchmarking

- ▶ Mismo prompt → múltiples proveedores en paralelo
- ▶ Calidad, costo y disponibilidad (SLA)
- ▶ Términos de licenciamiento y compliance
- ▶ FastAPI como gateway multi-proveedor

03

40 min

Agentes en Google Cloud

- ▶ Agent Studio: agente sin código
- ▶ Google ADK: agente en Python
- ▶ Despliegue a Cloud Run / Agent Runtime
- ▶ 7 criterios de selección de proveedor



Break 15 min entre Bloque 1 y 2

¿Por qué tantos Proveedores de LLM?

Tema 1 · Directo vs Hyperscaler — el dilema de cada arquitecto

El Dilema

- ✗ API directa: rápida de integrar, pero sin SLA contractual
- ✗ Hyperscaler: SLA y compliance, pero más fricción de setup
- ✗ Cada proveedor factura y limita distinto (rate limits, cuotas)
- ✗ Vendor lock-in si el código depende de un solo SDK
- ✗ Licencias distintas: ¿puedo auto-hospedar? ¿reentrenan con mis datos?



El Enfoque

- ✓ Evaluar con un framework de criterios objetivo (costo/calidad/SLA)
- ✓ Abstractar el proveedor detrás de un gateway propio (FastAPI)
- ✓ Usar Ollama local como fallback sin costo y sin dependencia externa
- ✓ Revisar licenciamiento ANTES de escalar (no después)
- ✓ Documentar el DPA y residencia de datos por proveedor



Anatomía de una Llamada de Inferencia

Tema 1 · De la API key a una respuesta del modelo



Llamada API — Patrón Universal

```
import os
from openai import OpenAI

cliente = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
resp = cliente.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}])
```

Parámetros clave

```
temperature=0.7, max_tokens=500

stream=True

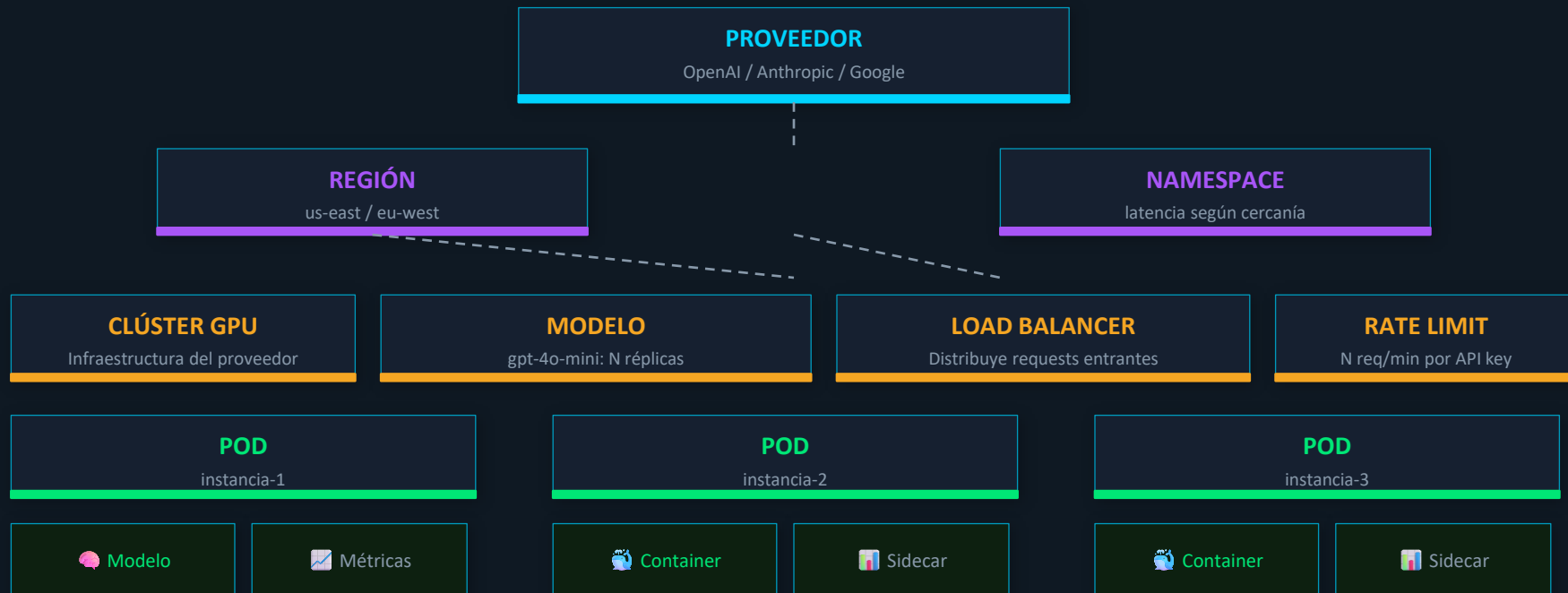
response_format={"type": "json_object"}

tools=[...] (function calling)
```

```
# Creatividad vs longitud de respuesta
# Respuesta token por token (SSE)
# Forzar salida JSON estructurada
# El modelo puede invocar funciones propias
```

Arquitectura de un Servicio de Inferencia

Tema 1 · De tu app a la infraestructura del proveedor



Ollama: Tu Proveedor de Inferencia Local

Tema 1 · El único "proveedor" sin licenciamiento de API ni costo por token



Ollama — Setup en 3 pasos

```
# Instalar Ollama

curl -fsSL https://ollama.ai/install.sh | sh

# Descargar modelo (2GB)

ollama pull llama3.2:3b

# Ejecutar y consultar

ollama serve

curl http://localhost:11434/api/chat \

  -d '{"model": "llama3.2:3b", ...}'
```

Modelos recomendados para clase:

llama3.2:3b 2GB ★ Recomendado aula

mistral:7b 4GB Buenas instrucciones

phi3:mini 2.3GB Rápido y eficiente



FastAPI Gateway — Endpoints

GET	/health	Estado del servicio
GET	/models	Modelos disponibles
POST	/chat	Chat con LLM
POST	/chat/stream	Streaming SSE
POST	/embeddings	Vectores semánticos

```
@app.post("/chat")
async def chat(req: ChatRequest):
    resp = await ollama_client.chat(
        model=req.model,
        messages=req.messages)
    return ChatResponse(**resp)
```

Mismo Prompt → Tres Proveedores

Tema 2 · Comparar respuestas y costos en paralelo



OpenAI

```
from openai import OpenAI

cliente = OpenAI(
    api_key=os.getenv("OPENAI_API_KEY"))

resp = cliente.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user",
               "content": prompt}])

print(resp.choices[0]
      .message.content)

# Costo: $0.15/$0.60 por MTok

# (input/output)
```



Prompt único

```
prompt = "Resume este contrato en 100  
palabras"
```



Anthropic

```
import anthropic

cliente = anthropic.Anthropic(
    api_key=os.getenv("ANTHROPIC_API_KEY"))

resp = cliente.messages.create(
    model="claude-3-haiku-20240307",
    max_tokens=200,
    messages=[{"role": "user",
               "content": prompt}])

print(resp.content[0].text)

# Costo: $0.25/$1.25 por MTok
```



AWS Bedrock

```
import boto3, json

cliente = boto3.client(
    "bedrock-runtime")

cuerpo = {
    "max_tokens": 200,
    "messages": [{"role": "user",
                   "content": prompt}]}

resp = cliente.invoke_model(
    body=json.dumps(cuerpo))
```

APIs Directas: OpenAI vs Anthropic vs Google

Tema 2 · Mismo patrón de código, distintos SDKs y precios

Criterio	OpenAI	Anthropic	Google AI
Modelo flagship	GPT-4o	Claude Sonnet	Gemini 2.5 Pro
\$/MTok input	\$2.50	\$3.00	\$1.25
\$/MTok output	\$10.00	\$15.00	\$5.00
Contexto máx.	128K tokens	200K tokens	1M-2M tokens
Function calling	Sí, maduro	Sí, tool use	Sí, nativo
Tier gratuito	\$5 crédito inicial	Sin tier gratis	Generoso (AI Studio)
SLA contractual	No (usar Azure)	No (usar Bedrock)	No (usar Vertex AI)

💡 Para SLA contractual y compliance empresarial, usa el mismo modelo vía Azure OpenAI, AWS Bedrock o Vertex AI en vez de la API directa

Cliente Unificado — Un Solo Wrapper Python

Tema 2 · Abstrae el proveedor detrás de una interfaz común

clientes/llm_router.py

```
def llamar_modelo(proveedor, prompt):  
    if proveedor == "openai":  
        return llamar_openai(prompt)  
    elif proveedor == "anthropic":  
        return llamar_anthropic(prompt)  
    elif proveedor == "ollama":  
        return llamar_ollama(prompt)  
    else:  
        raise ValueError(f"Proveedor desconocido")
```

Fallback Automático

```
try:  
    resp = llamar_modelo("openai", prompt)  
except RateLimitError:  
    # Fallback a proveedor secundario  
    resp = llamar_modelo("anthropic", prompt)
```

CLI — Comandos esenciales

<code>python ejemplos/05_benchmark_multiproveedor.py</code>	← Comparar
<code>uvicorn main:app --reload</code>	← Backend
<code>ollama pull llama3.2:3b</code>	← Modelo local
<code>curl http://localhost:8000/proveedores</code>	← Credenciales
<code>npm run dev</code>	← Frontend
<code>adk web</code>	← Agente ADK



Todas las credenciales se leen con `os.getenv()` desde `.env` — nunca hardcodeadas en el código

7 Criterios para Evaluar un Proveedor de Inferencia

Tema 3 · Framework de decisión para seleccionar proveedor de LLM



01

25%

Costo por Token

Input + output + fine-tuning + embeddings. Proyección a volumen real de uso.



02

20%

Calidad de Respuesta

Benchmarks (MMLU, HumanEval), razonamiento, alucinaciones, soporte multilingüe.



03

15%

Disponibilidad (SLA)

Uptime contractual vs histórico, rate limits, status page, incident history.



04

15%

Licenciamiento & Uso de Datos

¿Entrena con tus datos? Retención, DPA, residencia geográfica, AUP.



05

10%

Ecosistema & Tooling

SDKs oficiales, function calling, frameworks (LangChain, ADK), observabilidad.



06

10%

Cumplimiento & Seguridad

SOC2, ISO27001, GDPR/LGPD, Zero Data Retention, cifrado en tránsito y reposo.



07

5%

Vendor Lock-in

Facilidad de migrar a otro proveedor, SDKs propietarios vs estándares abiertos.

Análisis de Costos: 1M Tokens/Mes

Tema 3 · Escenario real: chatbot de soporte con volumen medio

Componente	OpenAI	Anthropic	Google AI	Ollama Local
Tokens input (750K)	\$1.88	\$2.25	\$0.06	\$0
Tokens output (250K)	\$2.50	\$3.75	\$0.08	\$0
Infraestructura propia	\$0	\$0	\$0	\$45
Fine-tuning (opcional)	\$25	\$0	\$0	\$0
Embeddings	\$10	\$0	\$8	\$0
Soporte/SLA	incluido	no disp.	no disp.	manual
TOTAL/MES	\$39.63	\$6.00	\$8.14	~\$45

ROI y Gestión de Riesgos de Proveedor

Tema 3 · Qué evaluar antes de comprometerse con un proveedor



Retorno de Inversión

312%

ROI a 12 meses

2.1m

Payback period

\$52K

Beneficio neto anual

Beneficios cuantificados (equipo de soporte de 20 personas):

Reducción tickets de soporte L1	+\$32K/año
Clasificación automática de tickets	+\$18K/año
Menor error en extracción de datos	+\$9K/año
Time-to-respuesta -60%	+\$14K/año
Costo de API vs contratación adicional	-\$21K/año



Riesgos y Mitigaciones

ALTO

Vendor Lock-in

Wrapper propio + soporte multi-proveedor desde el día 1

ALTO

Costo escalado sin control

Budget alerts + rate limiting + caché de respuestas

MED

Cambios de precio del proveedor

Contratos con precio fijo o Reserved Capacity

ALTO

Datos sensibles en prompts

Zero Data Retention + anonimización PII antes del envío

MED

Alucinaciones LLM

RAG + guardrails + validación humana

MED

Disponibilidad API

Fallback Ollama local + circuit breaker

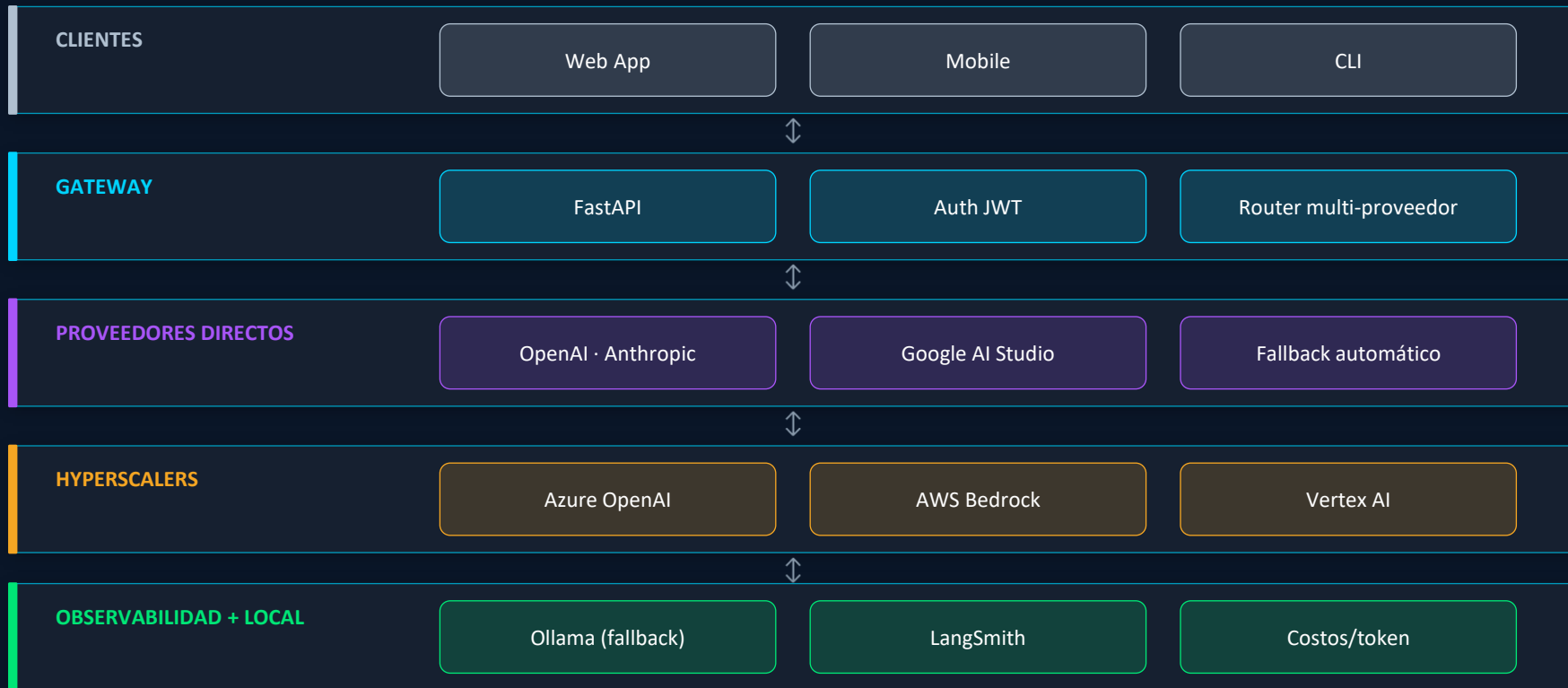
BAJO

Cambios en términos de licencia

Revisión legal periódica + alertas de ToS

Arquitectura de Referencia: Multi-Proveedor

Visión integral — de tu app a múltiples proveedores con fallback



Agentes en Google Cloud — Parte 3.2: ADK

CON CÓDIGO

Tema 3 · Google Agent Development Kit — control total en Python



ADK — agent.py

```
from google.adk.agents import Agent

from google.adk.tools import FunctionTool

def consultar_faq(tema: str) -> str:

    return FAQ_TIENDA.get(tema, "N/A")

root_agent = Agent(

    name="asistente_tienda",

    model="gemini-2.0-flash",

    tools=[FunctionTool(consultar_faq)])
```

Comandos CLI del ADK:

adk web ← UI de desarrollo local

adk run adk_agent ← terminal interactiva

adk deploy cloud_run ← despliegue a producción

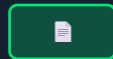


Estructura del proyecto



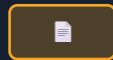
adk_agent/

carpeta raíz del agente



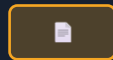
agent.py

define root_agent + tools



__init__.py

expone el agente al ADK



requirements.txt

google-adk, python-dotenv



Cloud Run

destino final del despliegue

```
# Ideal para:
# Desarrolladores, lógica custom,
# integración con sistemas propios
```

Agentes en Google Cloud — Parte 3.1: Consola

SIN CÓDIGO

Tema 3 · Crear un agente sin escribir código — Agent Studio



Agent Studio — 3 pasos

1. Habilitar APIs

```
gcloud services enable aiplatform.googleapis.com
```

2. Crear agente en la consola

```
console.cloud.google.com → Agent Studio
```

3. Definir instrucciones + Data Store

Instructions: rol + tono + límites

Data Store: FAQs, PDFs, texto plano

→ respuestas basadas en RAG

Ejemplo: Asistente de FAQ de tienda

Horario: L-S 9-20h

Pagos: Tarjeta, transferencia, y efectivo

Devoluciones: 30 días con boleta



Flujo en Agent Studio

1

Crear agente

Nombre, idioma, región

2

Configurar

Instrucciones + modelo base

3

Data Store

Cargar FAQs / documentos

4

Preview

Probar el agente en vivo

5

Integrar

Widget web / API REST

Ideal para:

Prototipos rápidos, analistas

no-developers, PoC de negocio



Resumen de la Sesión 3



Panorama: OpenAI, Anthropic, Google, Meta, Mistral, Azure, Bedrock, Vertex AI



Comparativa: calidad, costo por token y disponibilidad (SLA)



Licenciamiento: uso de datos, retención, DPA y AUP por proveedor



7 criterios de selección de proveedor de inferencia



Costos reales: OpenAI \$39.63 · Anthropic \$6.00 · Google \$8.14 · Ollama ~\$45/mes



Agentes en GCP: Agent Studio (sin código) y ADK (con código)



Recursos del Curso

Repositorio GitHub: github.com/bsg-institute/nivelacion-llm

Setup rápido: session_3/docs/SETUP.md

Comparativa costos: [ejemplos/02_comparativa*.py](#)

Benchmark multi-proveedor: [ejemplos/05_benchmark*.py](#)